Addition cancellation

MODEL $x, y \in \mathbb{F}_{2^{16}}$

$$(x + y) + y = x$$

Refinement map

$\alpha = \texttt{GF16.toGF216}$

Operation specs preserve $+$

α injective: model equality implies concrete equality

CONCRETE Aeneas-translated Rust `GF16`

$$\underbrace{\text{add } a\ b \gg\!= (\lambda s.\ \text{add } s\ b)}_{\text{execute the two additions}} = \text{ok}(a)$$

`add_add_cancel`: succeeds and returns exactly a.

ok(r) means **successful execution returning the value** r.

Distributivity

MODEL $x, y, z \in \mathbb{F}_{2^{16}}$

$$x \cdot (y + z) = x \cdot y + x \cdot z$$

Refinement map

$\alpha = \texttt{GF16.toGF216}$

Operation specs preserve $+$ and $\cdot$

α injective: model equality implies concrete equality

CONCRETE Aeneas-translated Rust `GF16`

$$L := \text{add } b\ c \gg\!= (\lambda s.\ \text{mul } a\ s)$$

$$R := \text{mul } a\ b \gg\!= (\lambda u.\ \text{mul } a\ c \gg\!= (\lambda v.\ \text{add } u\ v))$$

$$\exists r,\quad (L = \text{ok}(r)) \wedge (R = \text{ok}(r))$$

`mul_add_distrib`: both succeed with the same concrete value.

ok(r) means **successful execution returning the value** r.

Five key statements behind the examples

Existing operation specifications

`add_spec` : $\exists r,\ (\texttt{add}\ a\ b = \mathrm{ok}(r)) \wedge (\alpha(r) = \alpha(a) + \alpha(b))$

`mul_spec` : $\exists r,\ (\texttt{mul}\ a\ b = \mathrm{ok}(r)) \wedge (\alpha(r) = \alpha(a) \cdot \alpha(b))$

New proof: the existing interpretation map is injective

`toGF216_injective`

$$\alpha(u) = \alpha(v) \implies u = v$$

Specs + field laws + injectivity

New concrete results

`add_add_cancel`: adding b twice succeeds and returns a.

`mul_add_distrib`: both expressions succeed with the same value.

$\mathrm{ok}(r)$ means **successful execution returning the value** r.

$\alpha =$ `GF16.toGF216`. Cancellation uses `add_spec` and injectivity; distributivity also uses `mul_spec`.

These are the presentation's key statements; underlying arithmetic and execution lemmas are omitted.

Why start with smaller examples?

Starting point: SPQR verification

ML-KEM and full erasure coding were too complex
for our first educational bridging examples.

Too much implementation detail before the bridge becomes clear.

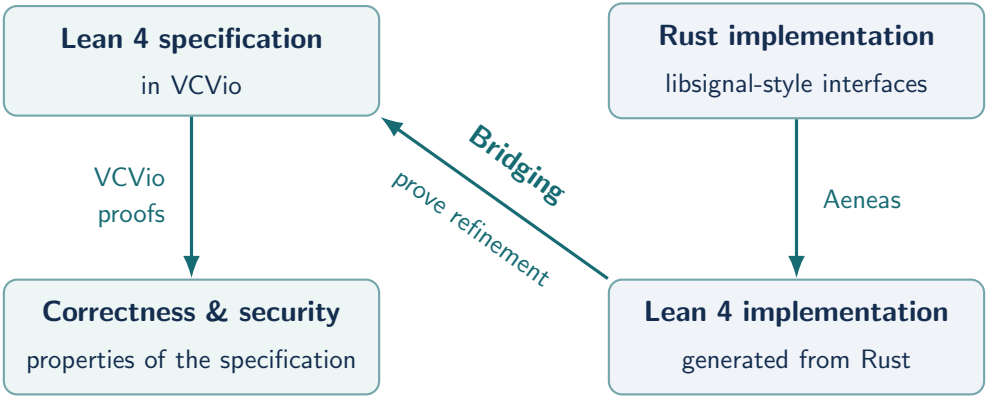
Smaller examples, the same verification task

Lazy vs. eager sampling and **commitment schemes**

Write Rust with libsignal-style interfaces, extract with Aeneas,
and connect the result to existing VCVio specifications and proofs.

Goal: teach the bridge from implementation to specification.

Bridging implementation and specification



The bridge lets specification-level guarantees apply to the implementation.

Representation maps and RNG/oracle assumptions are explicit.